PRÉ-PAPER · RASCUNHO

Programação Agêntica no Espaço de Pesos como Alternativa ao Fine-Tuning em Modelos de Linguagem Pequenos

Autores: Luiz Melo · **Afiliação:** Universidade do Sul de Santa Catarina (Unisul) · **Rascunho:** 9 nov. 2026

Rascunho de trabalho – o estado de cada resultado e o trabalho restante estão listados no final.

TL;DR

- **O defeito.** Modelos pequenos que "pensam" antes de responder costumam ficar presos repetindo a mesma coisa: *"Espera, deixa eu conferir… Sim… Espera…"*. A cada 100 problemas de matemática do ensino fundamental, o Qwen3.5-0.8B fica sem espaço em 85 antes de escrever a resposta.
- **A correção.** Demos a agentes Claude os pesos do modelo e um pequeno orçamento de testes, e pedimos que resolvessem isso editando os pesos diretamente, sem treinamento. **Com quatro edições minúsculas, o modelo passou de 15% para 67% de acerto nos nossos problemas de prática, e de 8% para 47% em matemática de competição mais difícil, que nunca apareceu durante a busca.**
- **A comparação.** A edição supera a publicada em *Sparks of Weight-Space Programming*, supera proibir a palavra "Wait" e supera o fine-tuning. O fine-tuning chegou a 29%, usando 13× mais tempo de GPU. O modelo editado também escreve respostas 60% mais curtas, então sai mais barato de usar.
- **O que importou.** Dois agentes independentes encontraram a mesma correção. Um script com as mesmas ferramentas e o mesmo orçamento, mas sem o raciocínio de um agente, não encontrou. Toda nota rápida que demos aos agentes acabou sendo burlada, e só os testes reais os mantiveram honestos. Surpreendentemente, dar mais dados de interpretabilidade aos agentes piorou as correções que eles encontraram.
- **Onde o método para de funcionar.** Num segundo defeito, espalhado (seguir instruções de formato, IFEval), nem os agentes, nem um system prompt, nem o fine-tuning melhoraram o modelo. Todo candidato que parecia melhor nos testes de prática perdeu de 3 a 7 pontos em prompts novos. O método funciona quando o defeito é concentrado e o efeito é grande. Ele não acha efeitos menores que o próprio ruído de amostragem do teste.

Tudo rodou em uma única placa de vídeo de jogos (GTX 1660 SUPER, 6 GB).

Um modelo pequeno que entra em loop costuma ser tratado como um problema de treinamento: juntar exemplos de bom raciocínio e fazer fine-tuning. Funciona, mas para um modelo que entra em loop até juntar os exemplos sai caro, porque a maioria das tentativas nunca termina.

Recentemente, Takehi et al. mostraram outro caminho. Um modelo de fronteira (o GPT-6 "Astra") leu os pesos brutos deste mesmo modelo e editou 0,016% deles, e o acerto no MATH-500 foi de 7,9% para 48,1%. Queríamos saber três coisas:

1. **Isso se reproduz** numa montagem barata, e o que exatamente aquela edição fez?
2. **É mais do que um truque?** Um ajuste de decodificação ou um fine-tuning comum fariam o mesmo?
3. **O que, no processo do agente, realmente importa?**

O defeito: um modelo que não consegue parar de pensar

O Qwen3.5-0.8B "pensa" num rascunho antes de responder e encerra o raciocínio com um marcador especial, `</think>`. Com um orçamento de 12.288 tokens (cerca de 9.000 palavras), a maioria das respostas nunca chega lá.



Figura 1: Comprimento de cada uma das 48 respostas de prática. A maioria das respostas do modelo original bate no limite de espaço; com a edição do agente, a maioria termina cedo.

Mudar o prompt não ajuda. Um prompt de sistema pedindo que o modelo não se repita não faz melhor do que um prompt placebo sem sentido. Os dois chegam a *aumentar* um pouco a taxa de loop (31% e 30%, contra 23% sem prompt). O loop está nos hábitos do modelo, não nas instruções.

As regras do jogo

Configuração

- **Modelo:** Qwen3.5-0.8B, na mesma revisão do estudo original.
- **Tarefa:** matemática do ensino fundamental (GSM8K), com raciocínio ligado e orçamento de 12.288 tokens.

- **Edições permitidas:** no máximo 4. Cada edição escolhe um grupo de "neurônios" (colunas de uma matriz de saída em uma camada), uma direção no espaço interno do modelo e uma intensidade. Ela muda apenas o quanto esses neurônios escrevem naquela direção:

$$w_j \leftarrow w_j - \alpha\,(u^\top w_j)\,u, \qquad |\alpha| \le 10.$$

  Em palavras: $\alpha = 1$ apaga essa parte do que os neurônios escrevem, $\alpha > 1$ inverte e $\alpha < 0$ reforça. Cada edição é uma mudança de posto 1.

- **Os agentes recebem:** os pesos, um briefing escrito e um avaliador com orçamento fixo. O avaliador oferece 40 "notas rápidas", que medem o quanto o modelo *quer* se repetir sem deixá-lo escrever. Também oferece 6 "testes reais", em que o modelo editado de fato resolve 16 problemas de prática.

- **Não é permitido:** treinar, rodar o modelo fora do avaliador, editar as camadas de vocabulário, nem ler a edição do estudo original, as nossas anotações, o trabalho de outros agentes ou o código do avaliador. Essas regras não foram impostas por software: auditamos depois cada ação no registro de cada agente e não encontramos violações.

- **Agentes:** Claude Opus 5.5, um por execução, cerca de 2 horas cada. Algumas execuções também receberam um **atlas**: um mapeamento detalhado da atividade interna do modelo enquanto ele entra em loop.

- **Como julgamos:**

  - **Conjunto de prática:** confirmamos a escolha de cada agente em outros 48 problemas (dev48).
  - **Teste trancado:** rodamos uma única escolha pré-registrada por método, uma única vez, em 32 problemas lacrados.
  - **Matemática mais difícil:** medimos a transferência para 148 problemas de competição (MATH-500) que nenhuma busca tocou.

- **Comparações justas:** todo método responde aos mesmos problemas com as mesmas sementes aleatórias, então dá para contar, problema por problema, quem resolveu o quê.

Um agente consegue corrigir os loops editando pesos?

EXPERIMENTO 1 Conseguimos reproduzir a edição publicada?

A edição publicada muda só duas matrizes, nas camadas 15 e 19. Nós a recuperamos com exatidão. Cada mudança é uma única edição de posto 1 com intensidade 6: ela pega a parte da saída de 64 neurônios que está ao longo de uma "direção de repetição" e a inverte, deixando-a em $-5\times$ o tamanho original. As duas camadas usam quase a mesma direção, escrita por neurônios diferentes.

Na nossa montagem, a edição publicada leva o conjunto de prática de 15% para 46%, e a matemática mais difícil de 8% para 21%. Nossos números absolutos no MATH são menores que os do artigo porque o nosso orçamento de tokens é 12k, e não 32k. Todos os métodos aqui usam o mesmo orçamento.

EXPERIMENTO 2 Um agente Claude consegue encontrar a própria correção?

Nosso primeiro agente (a0) só tinha as notas rápidas. Ele encontrou uma edição diferente, que desencoraja o modelo de copiar "Yes", "?" e "Wait" em duas camadas de atenção, e empatou com a edição publicada no teste trancado (44% contra 41%).

Depois acrescentamos os 6 testes reais. **Dois agentes, trabalhando de forma independente, convergiram para a mesma correção, e ela é bem mais forte: 67% e 65% no conjunto de prática.**

agente	o que a edição faz ($\leq$ 4 edições de posto 1)	conjunto de prática
a1	enfraquece a vontade de dizer "Wait" em 3 neurônios (L17n1915, L18n881, L22n573) e reforça o sinal de `</think>` na última camada	**67%** (26 resgatados, 1 perdido)
a2	remove "Wait" de 3 neurônios (dois deles os mesmos do a1) e reforça `</think>` na camada 22	**65%** (24 resgatados, 0 perdidos)

Como funciona: os dois agentes leram os pesos "através do vocabulário". Para cada neurônio, perguntaram que palavra ele empurra o modelo a dizer. Foi assim que acharam neurônios que empurram "Wait". As anotações deles mostram que aprenderam com os próprios erros. Negar esses neurônios por completo fez o modelo dizer "wait wait wait…" e não resolver nada, o que lhes mostrou que esses neurônios funcionam ao contrário do que tinham suposto.

As duas partes só funcionam juntas. Nas nossas ablações, cada uma um teste real com 16 problemas:

- o reforço de `</think>` sozinho resolve 19%;
- a supressão de "Wait" sozinha resolve 44–50%;
- as duas juntas resolvem 69%, e quase nada fica sem espaço.

Um loop precisa de um caminho de volta ("Espera, deixa eu conferir") e de nenhuma saída. A edição fecha o primeiro e abre a segunda.

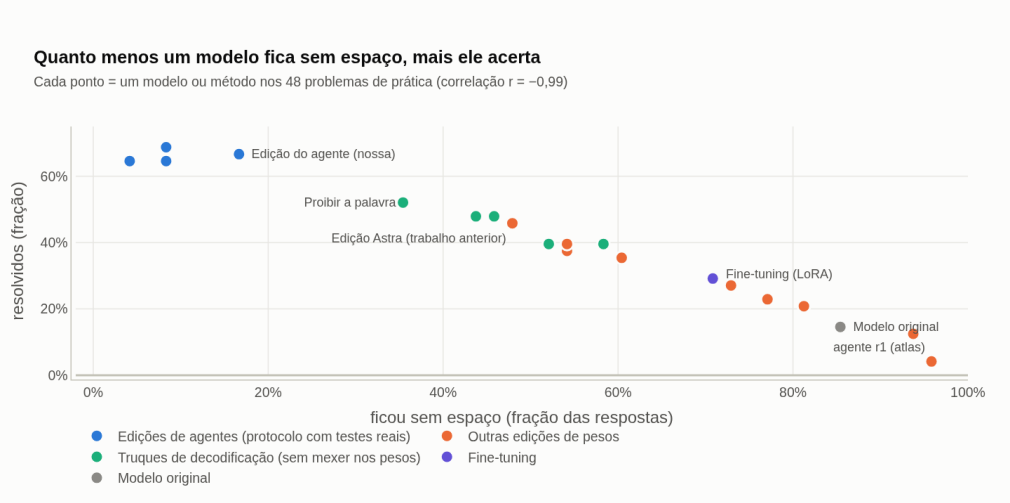


Figura 2: Nos 20 modelos e métodos que testamos, quanto menos respostas ficam sem espaço, mais respostas estão certas (r = −0,99).

EXPERIMENTO 3 **A correção se mantém em problemas que o agente nunca viu?**

No teste trancado, a edição do agente resolve 56% (o original: 13%; a edição publicada: 41%). Na matemática de competição, que nunca foi usada em nenhuma busca, ela resolve 47%, contra 21% da edição publicada. Comparada problema por problema com a edição publicada no MATH, ela resgatou 44 problemas e perdeu 6.

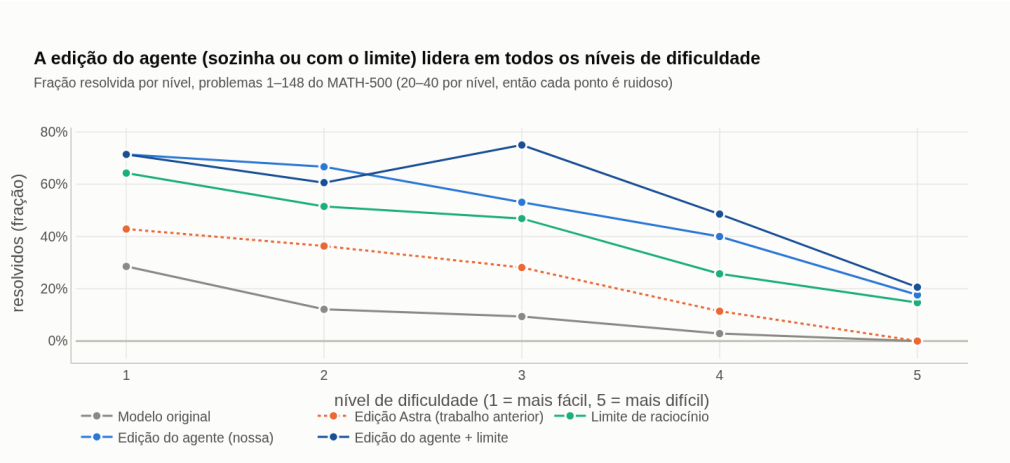


Figura 3: A edição do agente (sozinha ou com o limite de tempo de raciocínio) fica à frente em todos os níveis de dificuldade.

Com a edição, pensar volta a valer a pena: com 67%, o modelo editado supera o modelo original com o raciocínio desligado (cerca de 48%).

Um segundo teste trancado, maior, confirma. Abrimos mais 200 problemas lacrados uma única vez, depois de anotar as previsões. A edição do agente resolve 61,5% (o original: 15,5%; a edição publicada: 53,5%). Problema por problema, ela vence a edição publicada por **8,0 ± 3,4 pontos** (32 resgates, 16 perdas). Somando os 232 problemas trancados, a vantagem é de 9,1 ± 3,1.

É só um truque?

Um cético perguntaria se a edição é só um jeito sofisticado de fazer algo mais simples. Testamos as alternativas mais simples, ajustadas em 48 problemas (três vezes o que os agentes viram), o que as favorece.

método	prática (48)	teste trancado 1 (32)	teste trancado 2 (200)	matemática difícil (148)
modelo original	15%	13%	15,5%	8%
edição publicada (Astra)	46%	41%	53,5%	21%
proibir a palavra “Wait” (melhor de 3 ajustes)	52%	47%	—	27% (primeiros 48)
limite de raciocínio, 4.096 tokens (melhor de 2)	48%	**63%**	61,0%	37%
busca automática, sem agente	27%	—	—	—
fine-tuning (LoRA)	29%	—	—	—
edição do agente (a1)	**67%**	56%	61,5%	**47%**
edição do agente + limite de raciocínio	**69%**	**63%**	**69,5%**	**53%**

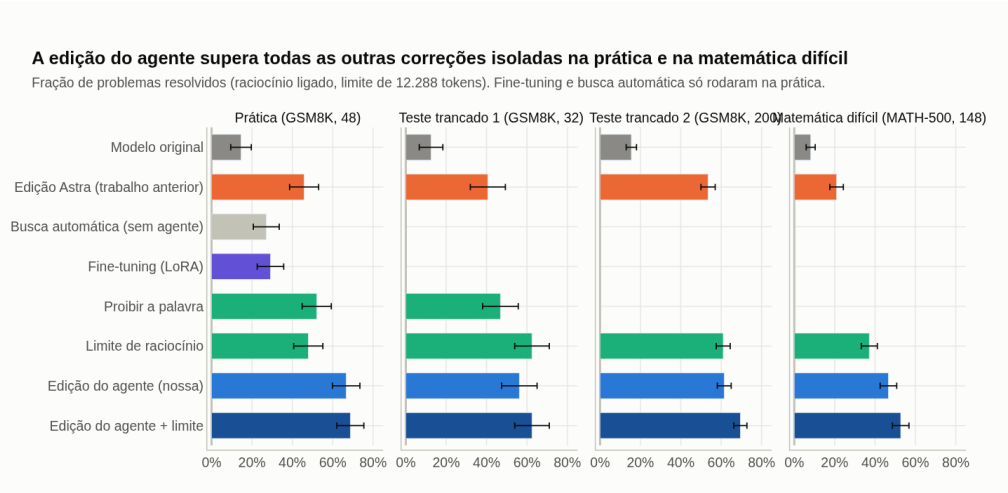


Figura 4: Parcela de problemas resolvidos por cada método nos quatro conjuntos de problemas (barras de erro: ±1 erro padrão).

EXPERIMENTO 4 Um ajuste de decodificação faria o mesmo?

Configuração

- **Proibir a palavra:** enquanto o modelo escreve, tornar "Wait" (e variantes) quase impossível e dar um empurrão em `</think>`. Nenhuma mudança nos pesos.
- **Limite de raciocínio ("budget forcing"):** depois de N tokens de raciocínio, cortar e forçar uma resposta. Nenhuma mudança nos pesos.

Proibir a palavra é pior em todos os casos. A edição não é uma proibição global. Ela age por meio de neurônios que disparam *quando o modelo está em loop*, então deixa "Wait" em paz quando ele é útil.

O limite de raciocínio é uma alternativa forte em matemática do ensino fundamental e empata com a edição nos testes trancados (61,0% contra 61,5% no teste de 200 problemas; +0,5 ± 3,4). Mas chega lá a um custo maior: o dobro de respostas bate no teto de tokens (34% contra 17%), e ele escreve cerca de 40% mais tokens. Proibir o "Wait" impede o modelo de recomeçar, mas não faz ele concluir. Em problemas mais longos ele atrapalha, porque também corta raciocínios que estavam indo bem. Na matemática mais difícil, a edição vence por 9,5 pontos, resgatando 23 problemas e perdendo 9. **Juntos, os dois são o melhor de tudo (53% na matemática mais difícil e 69,5% no teste trancado de 200 problemas, 8 pontos acima de cada um sozinho, com o menor número de respostas cortadas)**: a edição quebra os loops por dentro, e o limite pega o que sobra.

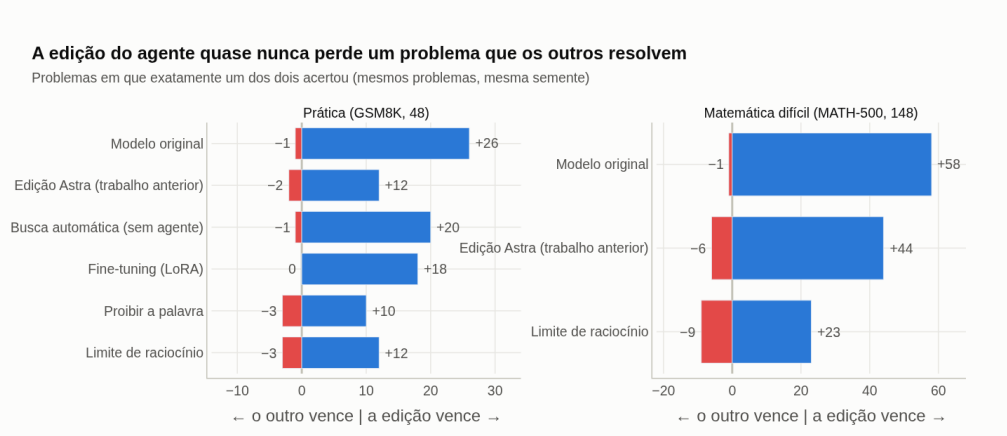


Figura 5: Problema por problema, com os mesmos problemas e sementes: problemas que só a edição do agente resolveu (direita) contra problemas que só o outro método resolveu (esquerda).

EXPERIMENTO 5 Um fine-tuning comum faria o mesmo?

Configuração

- **Dados:** o raciocínio do próprio modelo em 600 *outros* problemas do GSM8K. Só 57 tentativas terminaram de forma limpa e correta. Em 279 que entraram em loop, cortamos o raciocínio onde o loop começou, fechamos o raciocínio e deixamos o modelo responder: 118 respostas estavam certas. Isso dá 175 exemplos de treino.

- **Treinamento:** LoRA, um método de fine-tuning padrão e barato (posto 16, todas as camadas, 3 rodadas). Escolhemos a melhor rodada nos mesmos 16 problemas que os agentes usaram nos testes reais.

O fine-tuning chegou a 29% no conjunto de prática. É melhor que o original (15%), mas, problema por problema, a edição do agente resgatou 18 problemas que o fine-tuning errou e não perdeu nenhum. O modelo com fine-tuning ainda ficou sem espaço em 71% das respostas (a edição: 17%). Ele aprendeu o que escrever *depois* de um corte, não *quando* parar. Esta é uma receita padrão. Mais dados ou aprendizado por reforço poderiam fazer melhor, mas cada um custa mais tempo de GPU (veja abaixo).

EXPERIMENTO 6 O agente é necessário, ou bastam as ferramentas dele?

Escrevemos um script que testa edições da mesma caixa de ferramentas que os agentes vencedores usaram: "direções de palavras" para as 300 palavras mais comuns, os neurônios mais alinhados a elas e intensidades aleatórias. Ele teve o mesmo orçamento: 40 notas rápidas e mais 6 testes reais com os candidatos mais promissores.

O melhor teste real do script resolveu 3 de 16, e a escolha dele chegou a 27% no conjunto de prática, 40 pontos abaixo dos agentes. As ferramentas e o retorno não bastam. O que importa é o que o agente decide testar.

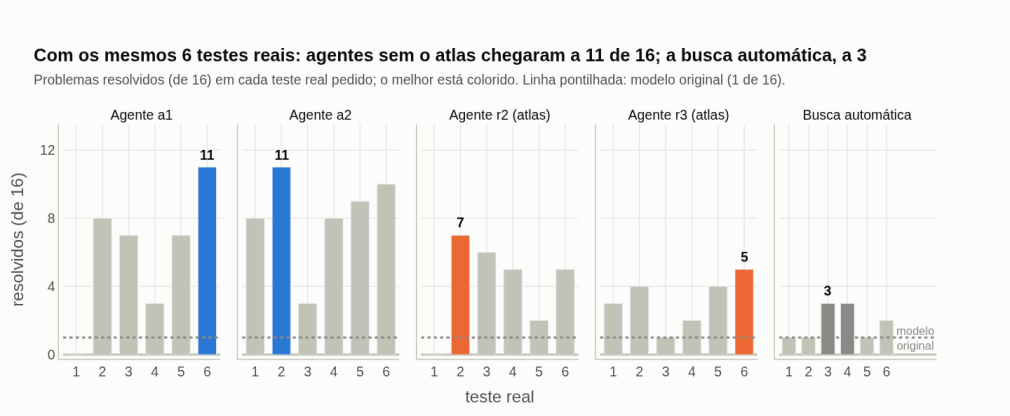


Figura 6: Os 6 testes reais de cada buscador (16 problemas cada). Os dois agentes sem o atlas chegaram a 11 de 16; os agentes com o atlas, a 5–7; o script, a 3.

O que aprendemos sobre o processo

EXPERIMENTO 7 Uma nota rápida pode substituir os testes reais?

Testes reais são lentos (16 problemas levam cerca de 10 minutos na nossa GPU), então a saída óbvia é um indicador rápido. Todo indicador que tentamos quebrou assim que um otimizador forte o pressionou:

- uma edição que pontuou melhor do que a edição publicada na "surpresa" do modelo com palavras de loop ganhou 6 pontos no uso real, contra 28 da edição publicada;
- uma edição sem nenhum loop palavra por palavra ainda ficou sem espaço em 77% das vezes;

- **o agente r1 levou a nota rápida a 20× o valor da edição publicada, e a edição dele resolveu 4% dos problemas, pior do que não fazer nada.** Ela bloqueou a cópia palavra por palavra, e o modelo simplesmente entrou em loop de outras formas (valores alternados, listas crescentes).

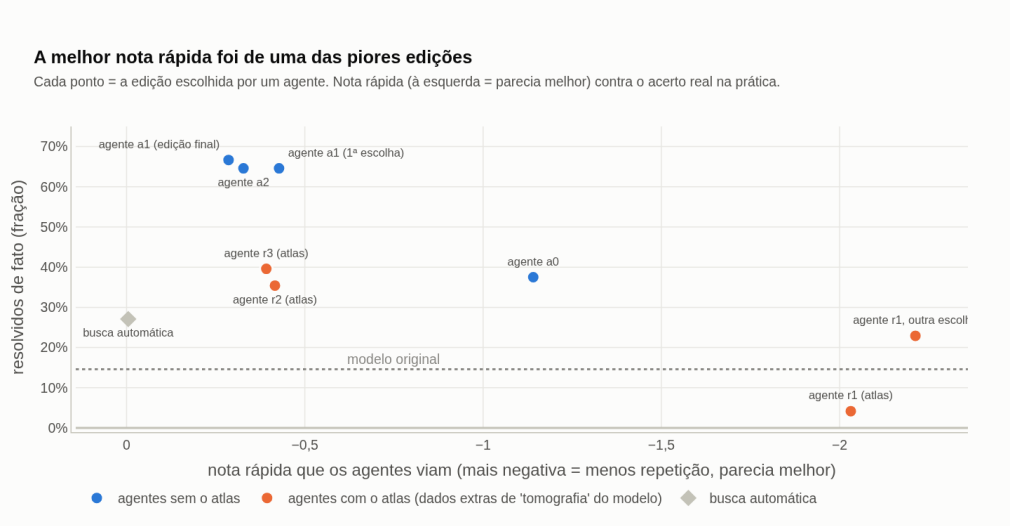


Figura 7: A nota rápida que a edição escolhida por cada agente obteve (à esquerda = parecia melhor) contra o quanto ela de fato resolveu problemas.

Seis testes reais resolveram isso. Nesse protocolo, as primeiras ideias de melhor nota dos dois agentes resolveram 0 de 16 num teste real, e os agentes as descartaram. **Nossa conclusão: o objetivo de um agente precisa vir do comportamento real, mesmo de uma amostra pequena dele. Indicadores servem como proteção, não como meta.**

EXPERIMENTO 8 Mais dados de interpretabilidade ajudam?

Rodamos três pares de agentes com instruções idênticas; em cada par, um dos agentes também tinha o atlas.

Nos três pares, o agente sem o atlas encontrou a edição melhor (38% contra 4%; 67% contra 35%; 65% contra 40%). Com o atlas, os agentes perseguiram mecanismos internos elaborados, como um estado de "reconferência" nas camadas do meio e "cabeças de loop". Pareciam convincentes nos dados, mas não paravam os loops. Sem ele, os agentes foram direto às alavancas: a palavra "Wait" e o sinal de `</think>`. Três pares tornam isso sugestivo, não comprovado.

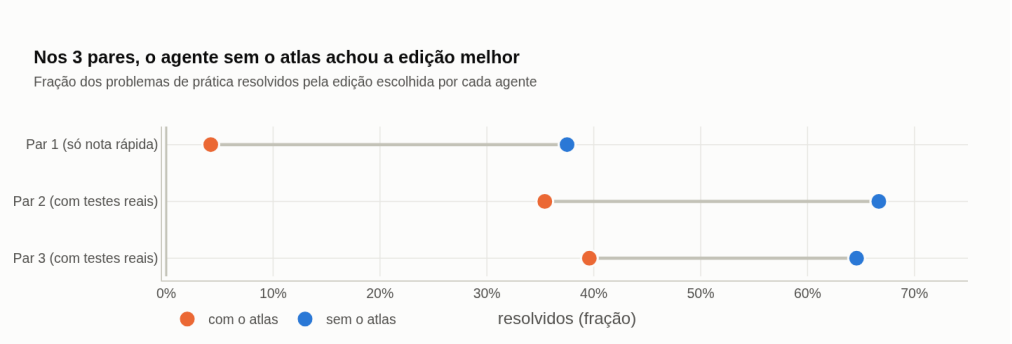


Figura 8: Cada par de agentes teve instruções idênticas; o agente sem o atlas foi melhor todas as vezes.

Quanto custou?

	prática	horas de GPU (1660 SUPER)	uso do agente
edição do agente (uma execução)	**67%**	**1,3**	5,4 M tokens lidos, 48 mil escritos (≈ US$ 5–7 a preço de API)
busca automática	27%	1,6	—
ajuste dos truques de decodificação	48–52%	4,4	—
fine-tuning (LoRA)	29%	17,1	—

- **Custo único.** Uma execução de agente levou cerca de 2 horas, o que dá mais ou menos a energia de um micro-ondas ligado por 15 minutos. O fine-tuning levou 17 horas de GPU, 10 delas só para gerar os dados de treino com um modelo que desperdiça a maior parte das tentativas.
- **Custo de uso.** As respostas do modelo editado são **60% mais curtas** no conjunto de prática e 29% mais curtas na matemática mais difícil. Essa economia se repete em cada pergunta respondida.
- **Comparação com o agente original.** A execução publicada usou 86,8 M tokens lidos e 2,9 M escritos ao longo de 20 horas (segundo os registros públicos). Nosso agente usou 16× e 60× menos, em cerca de 2 horas. As ferramentas são diferentes, então esta não é uma comparação controlada.
- **As letras miúdas.** Rodamos com uma assinatura, então os valores em dólar são estimativas a preço de tabela. Numa GPU alugada muito barata, o fine-tuning poderia custar alguns dólares a menos que o uso do agente, entregando menos da metade da melhoria.

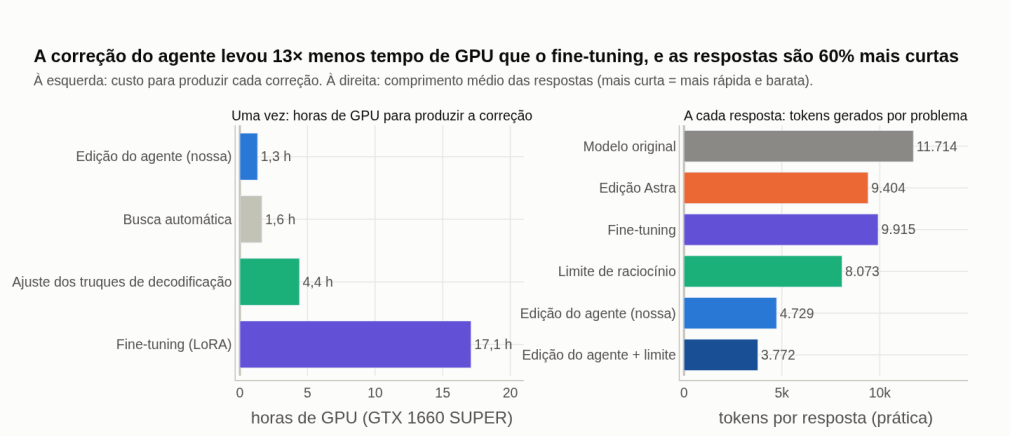


Figura 9: À esquerda, horas de GPU para produzir cada correção. À direita, o comprimento médio das respostas, que define o custo de cada resposta futura.

A cirurgia quebrou alguma coisa?

Com o raciocínio desligado, nenhuma das edições muda o acerto em matemática (200 problemas) ou em chamada de ferramentas (300 pedidos) além de cerca de dois

erros padrão. A edição age onde os loops acontecem, durante o raciocínio, e deixa as outras habilidades em paz. Há um efeito colateral positivo: com o raciocínio ligado, a edição publicada fica melhor em recusar chamadas de ferramenta irrelevantes (16 de 16 contra 11 de 16), provavelmente porque um raciocínio mais curto deixa menos espaço para inventar uma chamada.

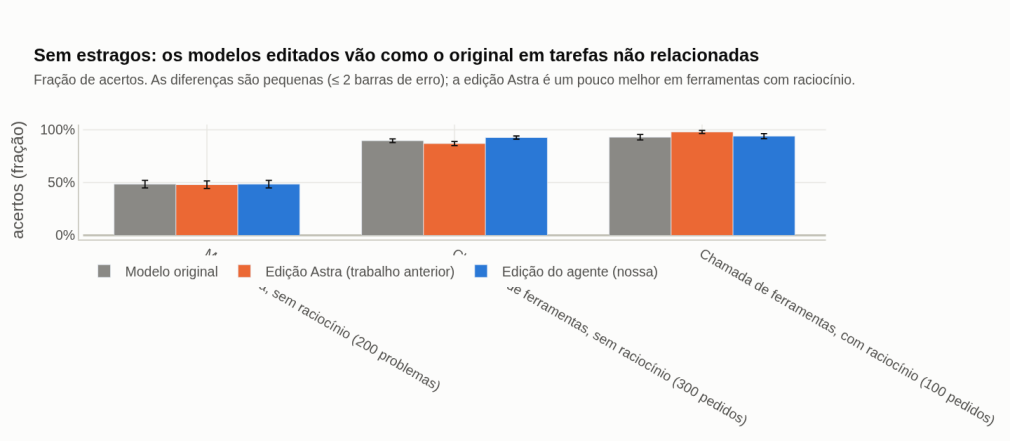


Figura 10: Acerto em tarefas não relacionadas para o modelo original, a edição publicada e a edição do agente.

Funciona num segundo defeito?

Não, e o jeito como falha é informativo. Precisávamos de um defeito sem palavra-chave óbvia. JSON e chamadas de ferramenta quebrados não eram um problema para este modelo com o raciocínio desligado: ele estava perto do teto. Escolhemos então **seguir instruções** (IFEval: "não use vírgulas", "tudo em minúsculas", "termine com esta frase exata", "destaque pelo menos 3 seções"). Com o raciocínio desligado, o modelo segue todas as instruções em só cerca de 55% dos prompts, e as falhas se espalham por muitos tipos de instrução.

Configuração

- **Mesmo protocolo dos loops:** um agente com 40 notas rápidas e 6 testes reais por geração, no máximo 4 edições de posto 1.
- **Comparações:** um system prompt pedindo cuidado, e fine-tuning LoRA em 414 respostas do próprio modelo que passaram no verificador oficial (a mesma receita de antes).
- **Confirmação:** em 152–200 prompts do IFEval nunca usados em nenhuma busca.

braço	mudança contra o original em prompts novos
agente, testes de 48 prompts (b1)	$-3{,}3 \pm 4{,}1$
a edição dos loops (a1), sem ajuste para isto	$-3{,}9 \pm 4{,}3$
system prompt	$-6{,}6 \pm 4{,}2$
fine-tuning (época escolhida em 48 prompts)	$-7{,}2 \pm 4{,}3$
agente, testes de 128 prompts (b2)	$-6{,}0 \pm 3{,}7$ (200 prompts)

Todo método pareceu melhor no próprio teste de prática (+5 a +12 pontos) e perdeu em prompts novos. A causa é o ruído de amostragem. Duas execuções do modelo *sem mudança nenhuma*, com sementes diferentes, discordam em 23% dos prompts. Em 48 prompts, isso dá uma margem de erro de 8–9 pontos, maior que qualquer ganho plausível. Um teste maior, de 128 prompts, também não salvou o agente: o controle de sinal oposto da melhor edição também pareceu levemente positivo.

O que isso significa. Nos loops, o efeito era de +46 pontos, e 16 problemas bastavam para vê-lo. Em seguir instruções, mudanças baratas de qualquer tipo (pesos, prompt, gradiente) parecem tirar o modelo de um ponto que já é quase o melhor da vizinhança, e nenhum teste desse tamanho acha as raras mudanças que ajudam. **O alcance do método é dado pela razão sinal/ruído do teste real, não só pelo agente.** O fine-tuning bate na mesma parede, então não é um caso de o agente perder para o gradiente.

O agente da segunda execução também achou, só lendo os pesos, uma direção do fluxo residual que 99,7% das linhas dos portões de esquecimento das 288 cabeças DeltaNet leem com o mesmo sinal: um canal global de controle do esquecimento. Editá-la muda o modelo, mas não muda o seguimento de instruções. Fica para trabalho futuro.

Contexto e trabalhos relacionados

Programação no espaço de pesos. Takehi et al. (2026) mostraram que modelos de fronteira conseguem ler e escrever pesos de redes neurais diretamente, incluindo a edição contra loops que reproduzimos. Usamos essa edição como referência e os registros públicos deles na comparação de custo.

Edição de modelos. Edições de posto 1 já foram usadas para mudar fatos armazenados num modelo (ROME, Meng et al., 2022; MEMIT, Meng et al., 2023). Apagar uma única direção pode remover um comportamento como a recusa (Arditi et al., 2024). Nossas edições pertencem a essa família. A diferença é que um agente as escolhe raciocinando sobre o comportamento, em vez de uma fórmula fixa.

Direcionamento (steering). Somar um vetor ao estado interno do modelo durante a execução pode direcionar o seu comportamento (ActAdd, Turner et al., 2023; engenharia de representações, Zou et al., 2023). Nossas edições são permanentes, em vez de aplicadas durante a execução.

Raciocínio excessivo. Modelos de raciocínio costumam reconferir mais do que precisam (Chen et al., 2024 [verificar]). Correções conhecidas incluem suprimir palavras como "Wait" (Wang et al., 2025 [verificar]) e limitar o tempo de raciocínio (Muennighoff et al., 2025). Usamos as duas como alternativas de comparação.

Agentes e Goodhart. Agentes já conduzem experimentos de interpretabilidade (MAIA, Shaham et al., 2024). Otimizadores fortes são conhecidos por burlar objetivos indiretos (Gao et al., 2023; Skalse et al., 2022). Vemos o mesmo com indicadores derivados de interpretabilidade.

O que ainda não podemos afirmar, e o que vem a seguir?

- **Tamanho das amostras:** as comparações principais agora se apoiam num teste trancado de 200 problemas ($\pm$3–4 pontos), mas os números de desenvolvimento e do MATH ainda se apoiam em 48–148 problemas.
- **Um modelo, um defeito que funciona:** "Wait" é uma palavra fácil de mirar, e o segundo defeito (IFEval) deu um resultado negativo bem controlado. **Próximo passo:** um defeito concentrado sem palavra-chave e um segundo modelo. Qualquer segundo defeito precisa de um teste real com ruído bem abaixo do efeito esperado, medido antes da busca.
- **Poucas réplicas:** o protocolo vencedor tem 2 agentes, e a comparação com e sem atlas tem 3 pares. **Próximo passo:** mais dois de cada.
- **Uma receita de fine-tuning:** uma receita mais forte poderia fechar parte da diferença, a um custo maior.
- **Regras por honra:** as restrições dos agentes foram auditadas, não impostas.
- **Amostrador diferente do estudo original:** os números absolutos mudam, mas toda comparação aqui usa as mesmas configurações.

A afirmação, como está hoje: para defeitos de comportamento concentrados e de efeito grande, um agente editando pesos pode ser uma alternativa mais rápida, mais barata e mais interpretável ao fine-tuning. Para defeitos espalhados, como seguir instruções num modelo de 0,8B, nem ele nem um fine-tuning barato ajudaram.

Dados, código e registros

Está tudo no repositório do projeto:

- **Caderno de laboratório:** datado, com as previsões escritas antes dos resultados (`docs/CADERNO.md`).
- **Registros de cada execução de agente:** briefings, anotações, registros do avaliador e cada edição submetida (267).
- **Figuras:** o notebook que desenha cada gráfico deste texto (`notebooks/results.ipynb`).
- **Custo:** um estudo detalhado (`docs/cost.md`).

As transcrições completas dos agentes e as gerações brutas (~210 MB) serão publicadas após revisão.

Estado deste rascunho e plano até a submissão

	estado
Reprodução e dissecação da edição publicada	feito
6 execuções de agente (2 protocolos, 3 pares com e sem atlas)	feito
Alternativas de decodificação, busca ingênua, fine-tuning	feito
Transferência para matemática mais difícil, efeitos colaterais, custo	feito
Teste trancado com $\geq$ 200 problemas	feito (t200: 5 métodos; faltam os braços “proibir Wait” e LoRA)
Mais réplicas (2 + 2 agentes)	a fazer
Segundo defeito sem palavra-chave (IFEval)	feito: negativo, com comparações de fine-tuning e de prompt
Segundo defeito concentrado; segundo modelo	a fazer (necessário para uma conferência principal)
Recorrigir o MATH com um verificador simbólico	a fazer
Conferir as citações; diagrama da anatomia da edição	a fazer

Onde publicar: primeiro um preprint no arXiv; depois um workshop de interpretabilidade ou de agentes de LLM com os resultados atuais; e uma conferência principal se um segundo defeito concentrado funcionar.

Títulos alternativos: *Agentes como Programadores do Espaço de Pesos: Corrigindo Loops de Raciocínio em Modelos de Linguagem Pequenos*; *Geração no Laço: o que Faz a Programação Agêntica no Espaço de Pesos Funcionar*.

Glossário

- **Modelo de linguagem, pesos:** o modelo é uma grande coleção de números (os *pesos*) organizados em matrizes. O Qwen3.5-0.8B tem cerca de 800 milhões deles. Treinar muda os pesos aos poucos, com gradientes. Aqui, um agente muda poucos pesos de forma direta e deliberada.
- **Token:** o pedaço de texto que o modelo lê e escreve de cada vez, cerca de ¾ de uma palavra.
- **Raciocínio (“thinking”), loop:** antes de responder, o modelo escreve um rascunho entre `<think>` e `</think>`. Ele está em *loop* quando repete o mesmo trecho (“Espera, deixa eu conferir…”) até esgotar o espaço.
- **Edição de posto 1 (rank-1):** a menor mudança estruturada numa matriz. Ela escolhe uma direção e um grupo de *neurônios* (colunas da matriz) e muda só o quanto esses neurônios escrevem naquela direção. Cada correção usou no máximo 4 dessas edições.
- **Fine-tuning, LoRA:** o caminho usual, que é treinar o modelo com exemplos. LoRA é a versão barata, que treina só pequenas matrizes acrescentadas ao modelo.

- **Steering (direcionamento):** somar um vetor às ativações internas para empurrar o comportamento. A edição Astra é desse tipo, só que gravada nos pesos.
- **Decodificação:** como o texto é sorteado a partir das probabilidades do modelo. *Proibir a palavra* (logit bias) e *limite de raciocínio* (budget forcing) mudam a decodificação, não os pesos.
- **Agente:** um modelo de IA (aqui, Claude) que trabalha sozinho com ferramentas: lê os pesos, propõe edições, pede avaliações e decide o próximo passo.
- **Nota rápida (sonda) e teste real:**
  - a nota rápida mede, em segundos, uma probabilidade interna (por exemplo, o quanto o modelo "quer" repetir);
  - o teste real faz o modelo editado de fato resolver problemas e confere as respostas. Lição central: notas rápidas podem ser burladas, e só o teste real é confiável.
- **Conjunto de prática, teste trancado, pré-registro:** os agentes só viam problemas de prática. O *teste trancado* fica lacrado até que cada método tenha escolhido sua versão final. As previsões foram escritas no caderno *antes* de abrir os resultados (pré-registro).
- **Erro-padrão ($\pm$ EP), comparação pareada:** "+8,0 $\pm$ 3,4" quer dizer um ganho de 8 pontos com incerteza de cerca de 3,4. Uma diferença maior que ~2 EP raramente é sorte. Na comparação pareada, os dois métodos fazem os *mesmos* problemas: *resgates* são os que só o primeiro acerta, e *perdas*, os que só o segundo acerta.
- **GSM8K, MATH-500, IFEval:** conjuntos públicos de avaliação.
  - GSM8K: matemática do ensino fundamental.
  - MATH-500: matemática de competição.
  - IFEval: seguir instruções verificáveis ("sem vírgulas", "tudo em minúsculas"…).
- **Lei de Goodhart:** quando uma medida vira meta, ela deixa de ser uma boa medida. É o que aconteceu com as notas rápidas.
- **Atlas:** um mapa extra da atividade interna do modelo (quais neurônios disparam em loops), dado a metade dos agentes.
- **DeltaNet:** o tipo de camada ("atenção linear") que forma 18 das 24 camadas do Qwen3.5. Ela guarda uma memória resumida do texto e tem *portões* que decidem o quanto esquecer.

Referências (a conferir e completar)

- Takehi, R., Teoh, J., Zhang, M., Khattab, O. (2026). *Sparks of Weight-Space Programming*. MIT CSAIL. https://rikiyat.github.io/blog/weight-space-programming/. Registros: https://github.com/RikiyaT/lm_understand-logs.
- Meng, K., Bau, D., Andonian, A., Belinkov, Y. (2022). Locating and Editing Factual Associations in GPT. NeurIPS.
- Meng, K., Sen Sharma, A., Andonian, A., Belinkov, Y., Bau, D. (2023). Mass-Editing Memory in a Transformer. ICLR.
- Arditi, A., et al. (2024). Refusal in Language Models Is Mediated by a Single Direction. NeurIPS.

- Turner, A. M., et al. (2023). Steering Language Models With Activation Engineering. arXiv.
- Zou, A., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv.
- Muennighoff, N., et al. (2025). s1: Simple Test-Time Scaling. arXiv.
- Chen, X., et al. (2024). Do NOT Think That Much for 2+3=? On the Overthinking of o1-like LLMs. arXiv. [verificar]
- Wang, C., et al. (2025). Wait, We Don't Need to "Wait"! Removing Thinking Tokens Improves Reasoning Efficiency. arXiv. [verificar]
- Shaham, T. R., et al. (2024). A Multimodal Automated Interpretability Agent. ICML.
- Gao, L., Schulman, J., Hilton, J. (2023). Scaling Laws for Reward Model Overoptimization. ICML.
- Skalse, J., et al. (2022). Defining and Characterizing Reward Hacking. NeurIPS.
- Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR.
- Cobbe, K., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv.
- Hendrycks, D., et al. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS.
- Lightman, H., et al. (2023). Let's Verify Step by Step. arXiv.

Anotações, dúvidas e sugestões